

《计算机系统基础》

第3章 程序的机器级表示 V：进阶话题

授课老师:

何璐璐

luluhe@whu.edu.cn

本讲内容

- 内存布局
- 缓冲区溢出
 - 漏洞
 - 保护

x86-64 Linux 内存布局

未按比例绘制

■ 栈

- 运行时栈(8MB限制)
- 例如，局部变量

■ 堆

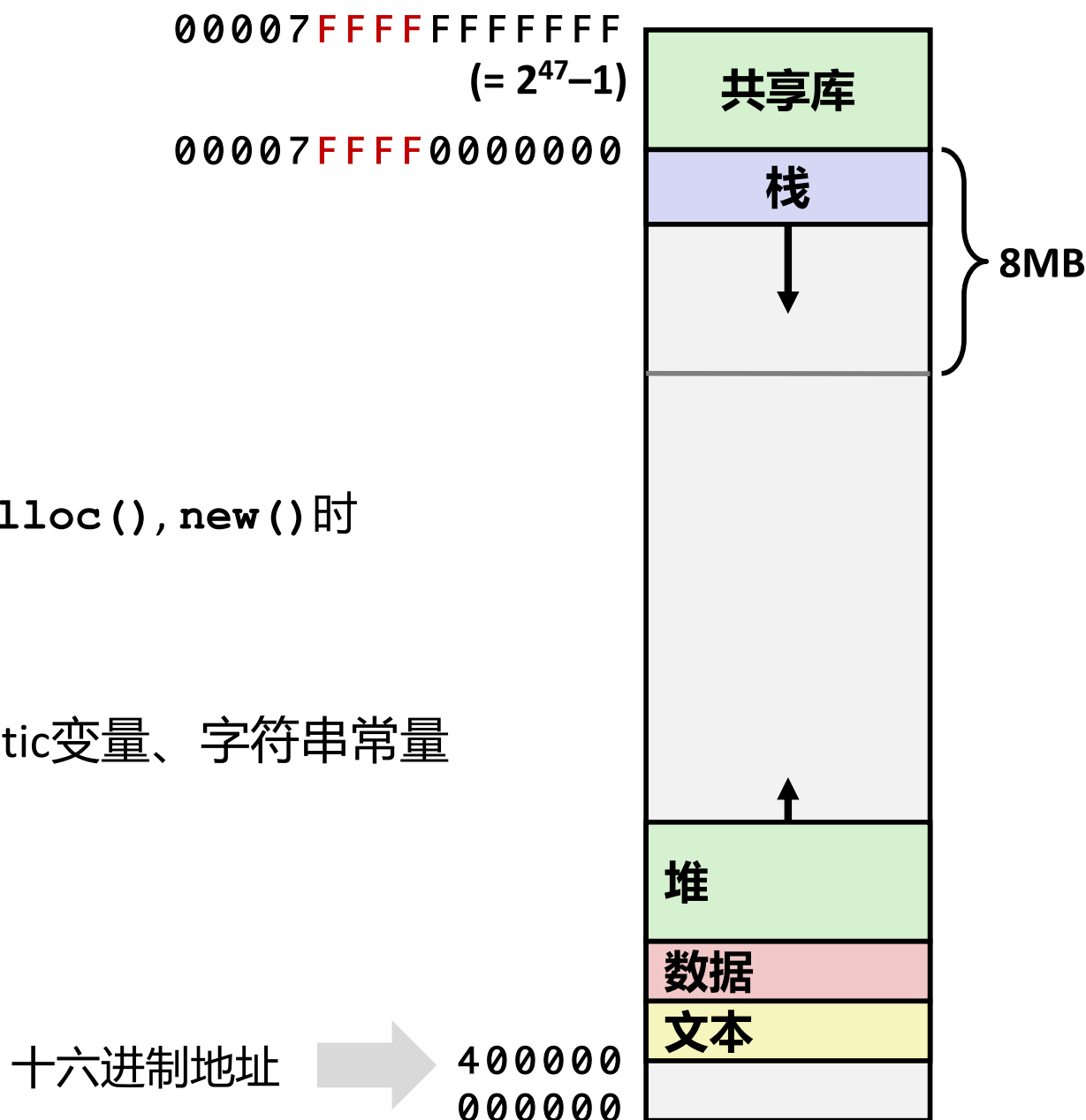
- 根据需要动态分配
- 当调用 `malloc()`, `calloc()`, `new()` 时

■ 数据

- 静态分配的数据
- 例如，全局变量、static变量、字符串常量

■ 代码段 / 共享库、

- 可执行的机器指令
- 只读



内存分配示例

未按比例绘制

00007FFFFFFFFFFFFFFF

```
char big_array[1L<<24]; /* 16 MB */
char huge_array[1L<<31]; /* 2 GB */

int global = 0;

int useless() { return 0; }

int main ()
{
    void *phuge1, *psmall2, *phuge3, *psmall4;
    int local = 0;
    phuge1 = malloc(1L << 28); /* 256 MB */
    psmall2 = malloc(1L << 8); /* 256 B */
    phuge3 = malloc(1L << 32); /* 4 GB */
    psmall4 = malloc(1L << 8); /* 256 B */
    /* Some print statements ... */
}
```

共享库

栈

堆

数据

文本

x86-64 示例地址

地址范围 $0 \sim 2^{47}$

```
local
phuge1
phuge3
psmall4
psmall2
big_array
huge_array
main()
useless()
```

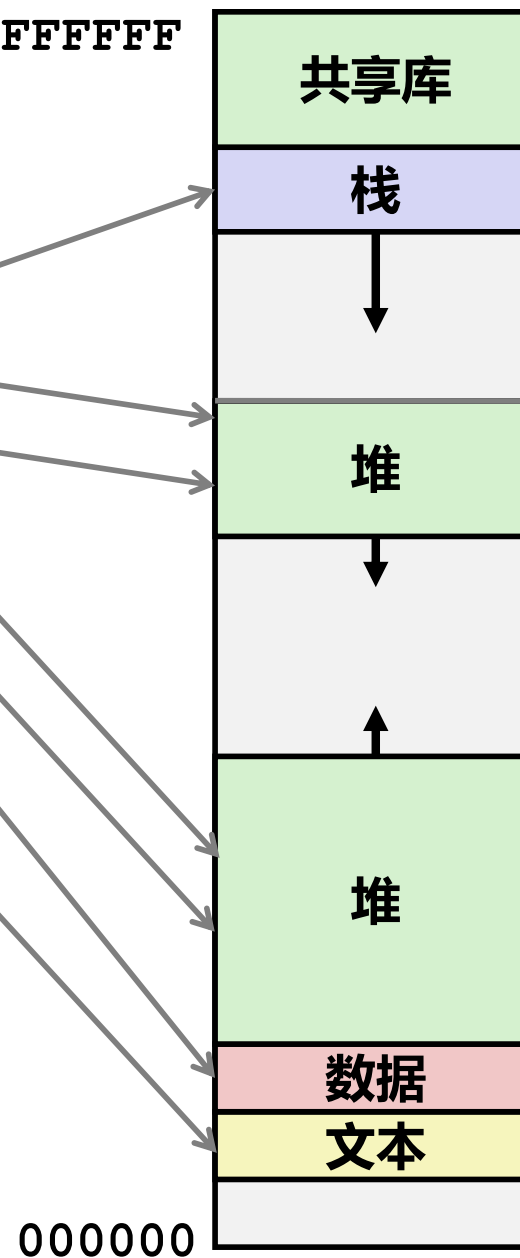
(具体值可能会因机器有所不同)

```
0x00007ffe4d3be87c
0x00007f7262a1e010
0x00007f7162a1d010
0x000000008359d120
0x000000008359d010
0x0000000080601060
0x0000000000601060
0x000000000040060c
0x0000000000400590
```

```
local      0x7ff7b2a06684
p1         0x7f8038f00000
p3         0x7f7f38f00000
p2         0x60000268c000
p4         0x60000268c100
big array  0x10d4fb020
huge array 0x10e4fb020
global     0x10d4fb018
useless    0x10d4f9dc0
main       0x10d4f9dd0
```

00007FFFFFFFFFFFFFFF

未按比例绘制

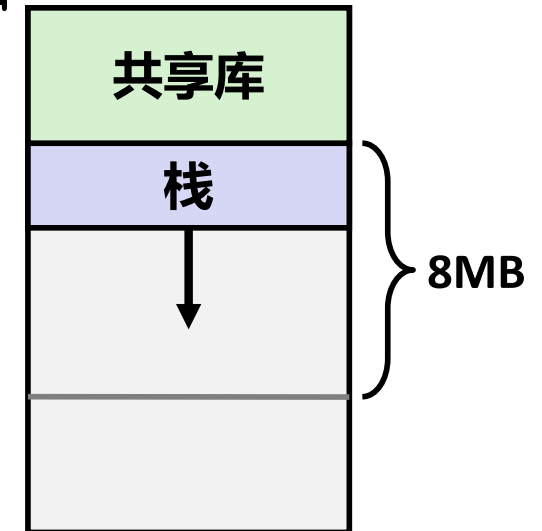


失控的栈 示例

未按比例绘制

00007FFFFFFFFFFFFF

```
int recurse(int x) {  
    int a[1<<15]; // 4*2^15 = 128 KiB  
    printf("x = %d.  a at %p\n", x, a);  
    a[0] = (1<<14)-1;  
    a[a[0]] = x-1;  
    if (a[a[0]] == 0)  
        return -1;  
    return recurse(a[a[0]]) - 1;  
}
```



- 函数在栈帧中存储局部数据
- 递归函数导致栈帧深度嵌套

本讲内容

- 内存布局
- 缓冲区溢出
 - 漏洞
 - 保护

回顾：内存引用Bug示例

```
typedef struct {  
    int a[2];  
    double d;  
} struct_t;  
  
double fun(int i) {  
    volatile struct_t s;  
    s.d = 3.14;  
    s.a[i] = 1073741824; /* 可能越界*/  
    return s.d;  
}
```

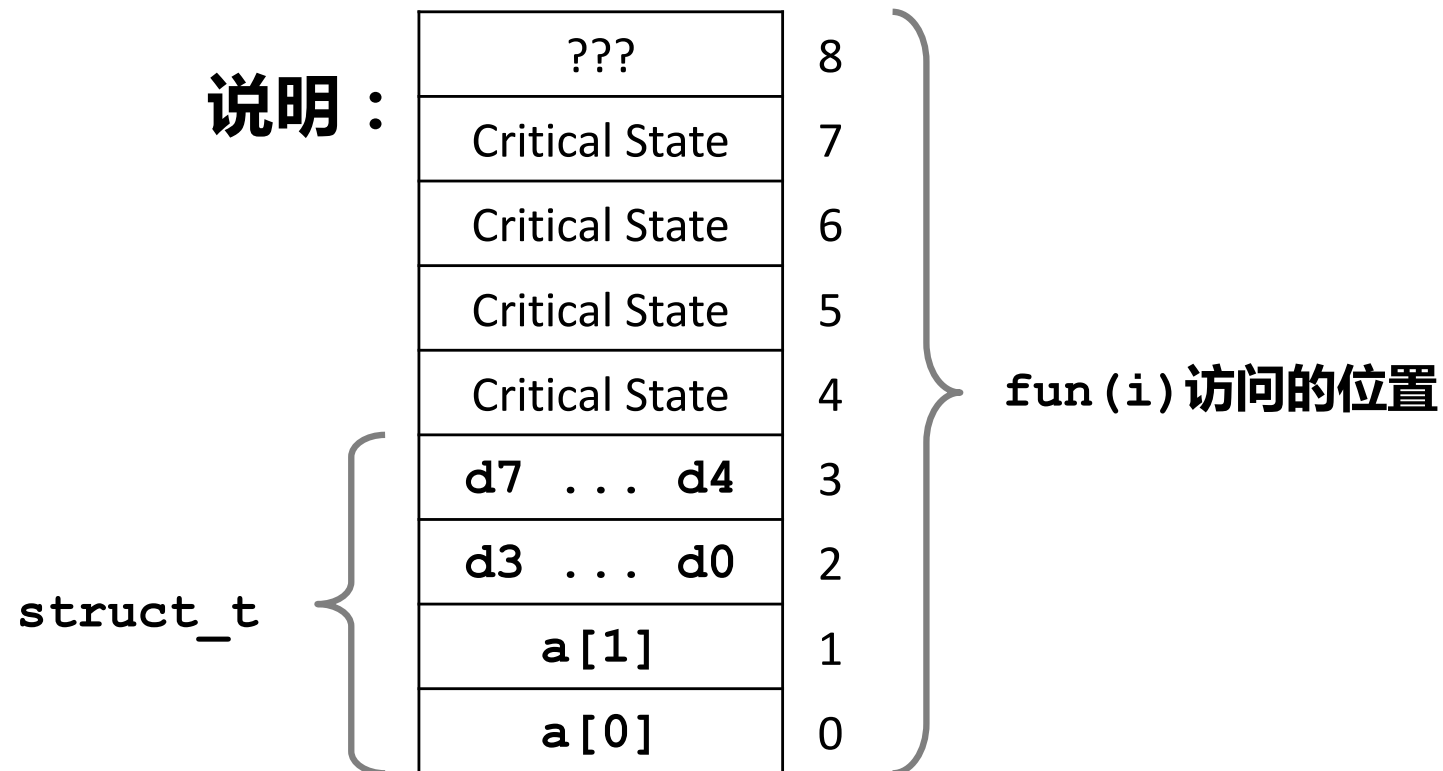
```
fun(0)    ->    3.1400000000  
fun(1)    ->    3.1400000000  
fun(2)    ->    3.1399998665  
fun(3)    ->    2.0000006104  
fun(6)    ->    Stack smashing detected  
fun(8)    ->    Segmentation fault
```

- 结果具有系统特异性

回顾：内存引用Bug示例

```
typedef struct {  
    int a[2];  
    double d;  
} struct_t;
```

fun(0)	->	3.1400000000
fun(1)	->	3.1400000000
fun(2)	->	3.1399998665
fun(3)	->	2.0000006104
fun(6)	->	Stack smashing detected
fun(8)	->	Segmentation fault



这些问题事关重大

■ 通常被称为“缓冲区溢出”

- 当超过为数组分配的内存大小时

■ 为什么事关重大？

- It's the #1 technical cause of security vulnerabilities
这是导致安全漏洞的#1技术原因
 - 主要原因是工程师/用户对其不够了解

■ 最常见的形式

- 未检查输入字符串的长度
- 特别是对于栈上有界的字符数组
 - 有时也被称为栈崩溃

String库源代码

■ Unix函数gets()的实现

```
/* Get string from stdin */
char *gets(char *dest)
{
    int c = getchar();
    char *p = dest;
    while (c != EOF && c != '\n') {
        *p++ = c;
        c = getchar();
    }
    *p = '\0';
    return dest;
}
```

- 没有办法对读取的字符数做限制

■ 其它库函数也有类似的问题

- **strcpy, strcat** : 复制任意长度的字符串
- **scanf, fscanf, sscanf** 当给定 %s 转换规范时

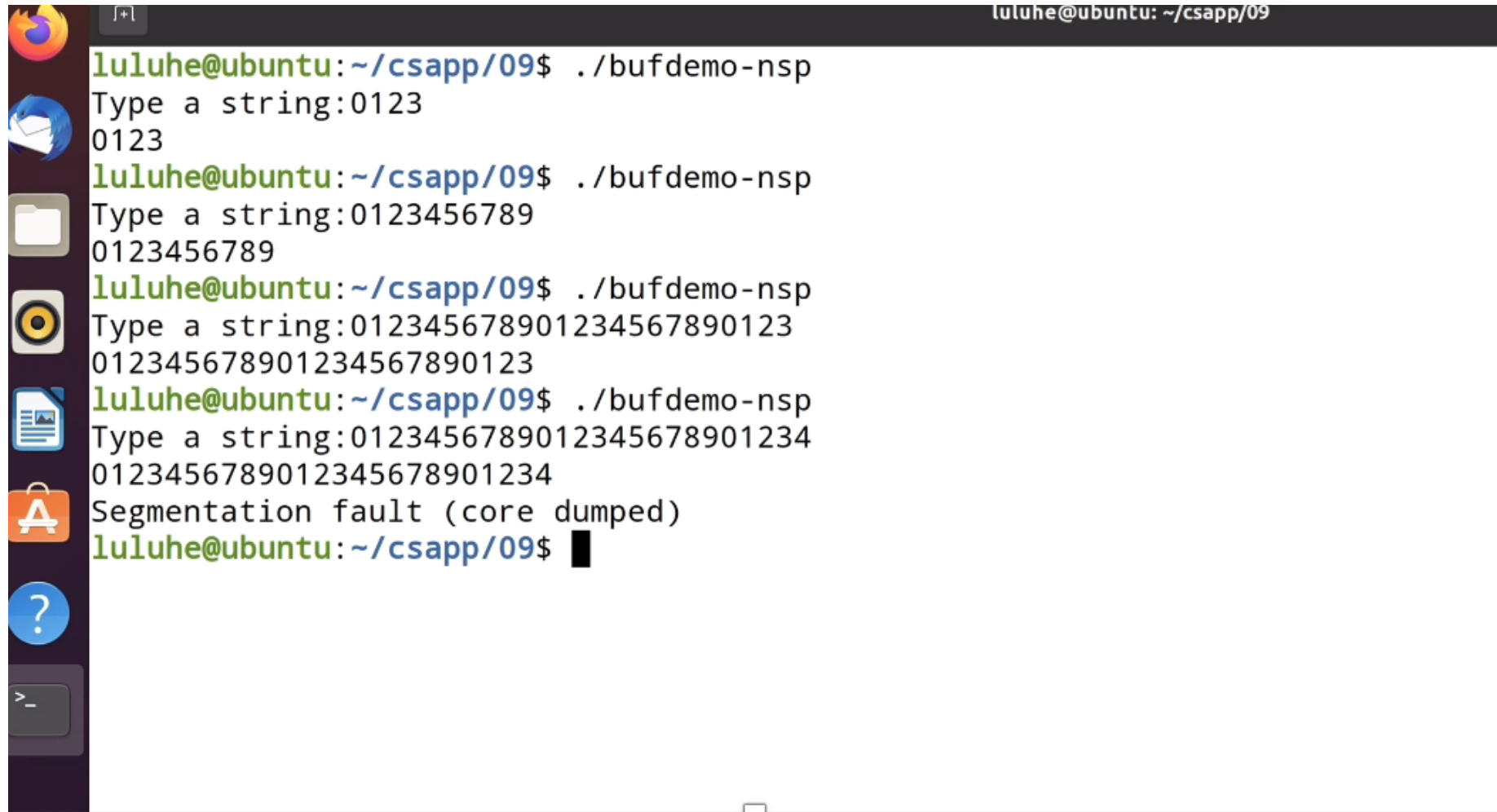
易受攻击的缓冲区代码

```
/* Echo Line */  
void echo()  
{  
    char buf[4];  /* Way too small! */  
    gets(buf);  
    puts(buf);  
}
```

```
void call_echo() {  
    echo();  
}
```

← 那么，
要多大才足够大呢

缓冲区溢出运行结果 示例

A terminal window titled 'luluhe@ubuntu: ~/csapp/09' showing the execution of a program named 'bufdemo-nsp'. The program prompts the user to 'Type a string:' and then prints the input. The first two runs show successful input of '0123' and '0123456789'. The third run shows a buffer overflow with the input '012345678901234567890123' being printed twice. The fourth run shows a segmentation fault (core dumped) after the input '0123456789012345678901234'.

```
luluhe@ubuntu:~/csapp/09$ ./bufdemo-nsp
Type a string:0123
0123
luluhe@ubuntu:~/csapp/09$ ./bufdemo-nsp
Type a string:0123456789
0123456789
luluhe@ubuntu:~/csapp/09$ ./bufdemo-nsp
Type a string:012345678901234567890123
012345678901234567890123
luluhe@ubuntu:~/csapp/09$ ./bufdemo-nsp
Type a string:0123456789012345678901234
0123456789012345678901234
Segmentation fault (core dumped)
luluhe@ubuntu:~/csapp/09$
```

缓冲区溢出汇编代码

echo:

000000000040069c <echo>:

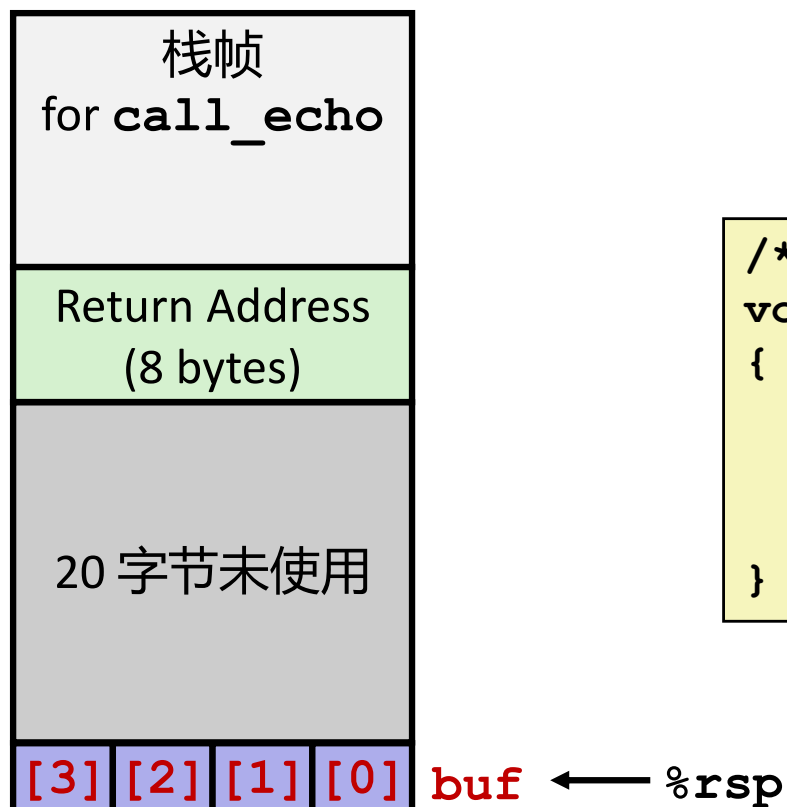
40069c:	48 83 ec 18	sub	\$0x18 , %rsp
4006a0:	48 89 e7	mov	%rsp , %rdi
4006a3:	e8 a5 ff ff ff	callq	40064d <gets>
4006a8:	48 89 e7	mov	%rsp, %rdi
4006ab:	e8 50 fe ff ff	callq	400500 <puts@plt>
4006b0:	48 83 c4 18	add	\$0x18, %rsp
4006b4:	c3	retq	

call_echo:

4006b5:	48 83 ec 08	sub	\$0x8, %rsp
4006b9:	b8 00 00 00 00	mov	\$0x0, %eax
4006be:	e8 d9 ff ff ff	callq	40069c <echo>
4006c3:	48 83 c4 08	add	\$0x8 , %rsp
4006c7:	c3	retq	

缓冲区溢出的栈示例

在调用gets之前



```
/* Echo Line */  
void echo()  
{  
    char buf[4]; /* Way too small! */  
    gets(buf);  
    puts(buf);  
}
```

```
echo:  
    subq $0x18, %rsp  
    movq %rsp, %rdi  
    call gets  
    . . .
```

缓冲区溢出的栈示例

在调用gets之前



```
void echo()  
{  
    char buf[4];  
    gets(buf);  
    . . .  
}
```

```
echo:  
    subq    $0x18, %rsp  
    movq    %rsp, %rdi  
    call    gets  
    . . .
```

call_echo:

```
. . .  
4006be:    callq    4006cf <echo>  
4006c3:    add      $0x8, %rsp  
. . .
```


缓冲区溢出的栈示例 #1

在调用gets之后

栈帧 call_echo			
00	00	00	00
00	40	06	c3
00	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
    subq    $0x18, %rsp
    movq    %rsp, %rdi
    call    gets
    . . .
```

call_echo:

```
. . .
4006be:    callq    4006cf <echo>
4006c3:    add     $0x8,%rsp
. . .
```

buf ← %rsp

输入23个字符

```
unix> ./bufdemo-nsp
Type a string: 01234567890123456789012
01234567890123456789012
```

```
"01234567890123456789012\0"
```

缓冲区溢出，但未损坏状态

缓冲区溢出的栈示例 #2

在调用gets之后

栈帧 call_echo			
00	00	00	00
00	40	00	34
33	32	31	30
39	38	37	36
35	34	33	32
31	30	29	28
27	26	25	24
23	22	21	20

buf ← %rsp

```
void echo()  
{  
    char buf[4];  
    gets(buf);  
    . . .  
}
```

call_echo:

```
. . .  
4006f1: callq 4006cf <echo>  
4006f6: add    $0x8,%rsp  
. . .
```

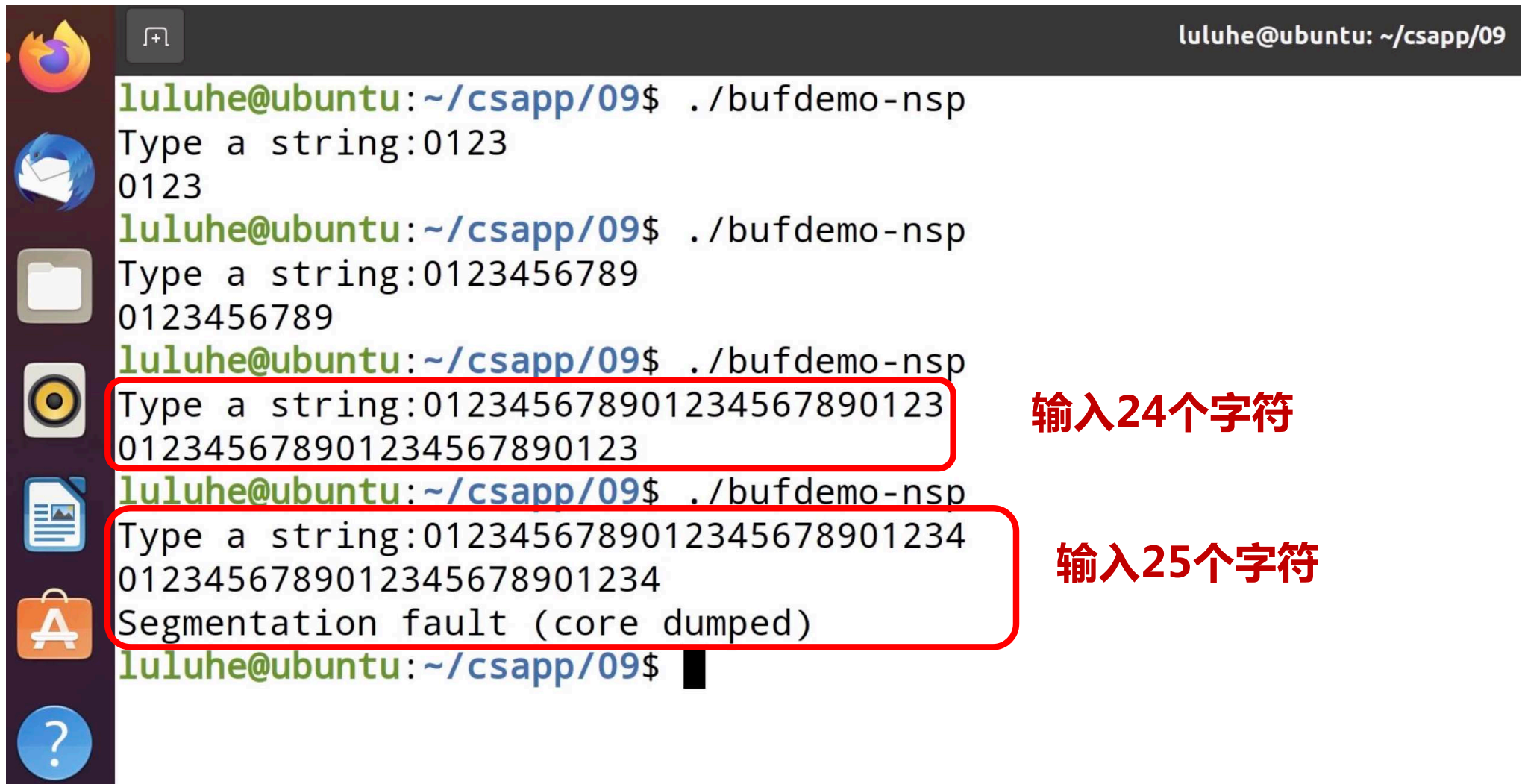
```
echo:  
    subq    $0x18, %rsp  
    movq    %rsp, %rdi  
    call    gets  
    . . .
```

输入25个字符

```
unix> ./bufdemo-nsp  
Type a string: 0123456789012345678901234  
Segmentation Fault
```

缓冲区溢出，破坏了返回指针

缓冲区溢出运行结果 示例

A terminal window with a dark background and a sidebar on the left containing icons for various applications. The terminal shows the execution of a program named 'bufdemo-nsp'. The user enters three different strings. The first two strings are '0123' and '0123456789', which are correctly processed. The third string is '0123456789012345678901234567890123', which is 24 characters long. This input is highlighted with a red box, and a red label '输入24个字符' (Input 24 characters) is placed to its right. The program prints the input string twice. The fourth input is '01234567890123456789012345678901234', which is 25 characters long. This input is also highlighted with a red box, and a red label '输入25个字符' (Input 25 characters) is placed to its right. After printing the input string twice, the program crashes with a 'Segmentation fault (core dumped)' message. The terminal prompt returns, and a black cursor is visible.

```
luluhe@ubuntu: ~/csapp/09$ ./bufdemo-nsp
Type a string:0123
0123
luluhe@ubuntu: ~/csapp/09$ ./bufdemo-nsp
Type a string:0123456789
0123456789
luluhe@ubuntu: ~/csapp/09$ ./bufdemo-nsp
Type a string:0123456789012345678901234567890123
0123456789012345678901234567890123
luluhe@ubuntu: ~/csapp/09$ ./bufdemo-nsp
Type a string:01234567890123456789012345678901234
01234567890123456789012345678901234
Segmentation fault (core dumped)
luluhe@ubuntu: ~/csapp/09$
```

输入24个字符

输入25个字符

缓冲区溢出的栈示例 #3

在调用gets之后

栈帧 call_echo			
00	00	00	00
00	40	06	00
33	32	31	30
39	38	37	36
35	34	33	32
31	30	29	28
27	26	25	24
23	22	21	20

buf ← %rsp

```
void echo()  
{  
    char buf[4];  
    gets(buf);  
    . . .  
}
```

call_echo:

```
. . .  
4006f1: callq 4006cf <echo>  
4006f6: add    $0x8,%rsp  
. . .
```

```
echo:  
    subq $0x18,%rsp  
    movq %rsp,%rdi  
    call gets  
    . . .
```

输入24个字符

为？什？么？

```
unix> ./bufdemo-nsp  
Type a string: 012345678901234567890123  
012345678901234567890123
```

缓冲区溢出，破坏了返回指针，但程序看起来还在工作！

缓冲区溢出的栈示例 #3 — 解释

在调用gets之后

栈帧 call_echo			
00	00	00	00
00	40	06	00
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ← %rsp

register_tm_clones:

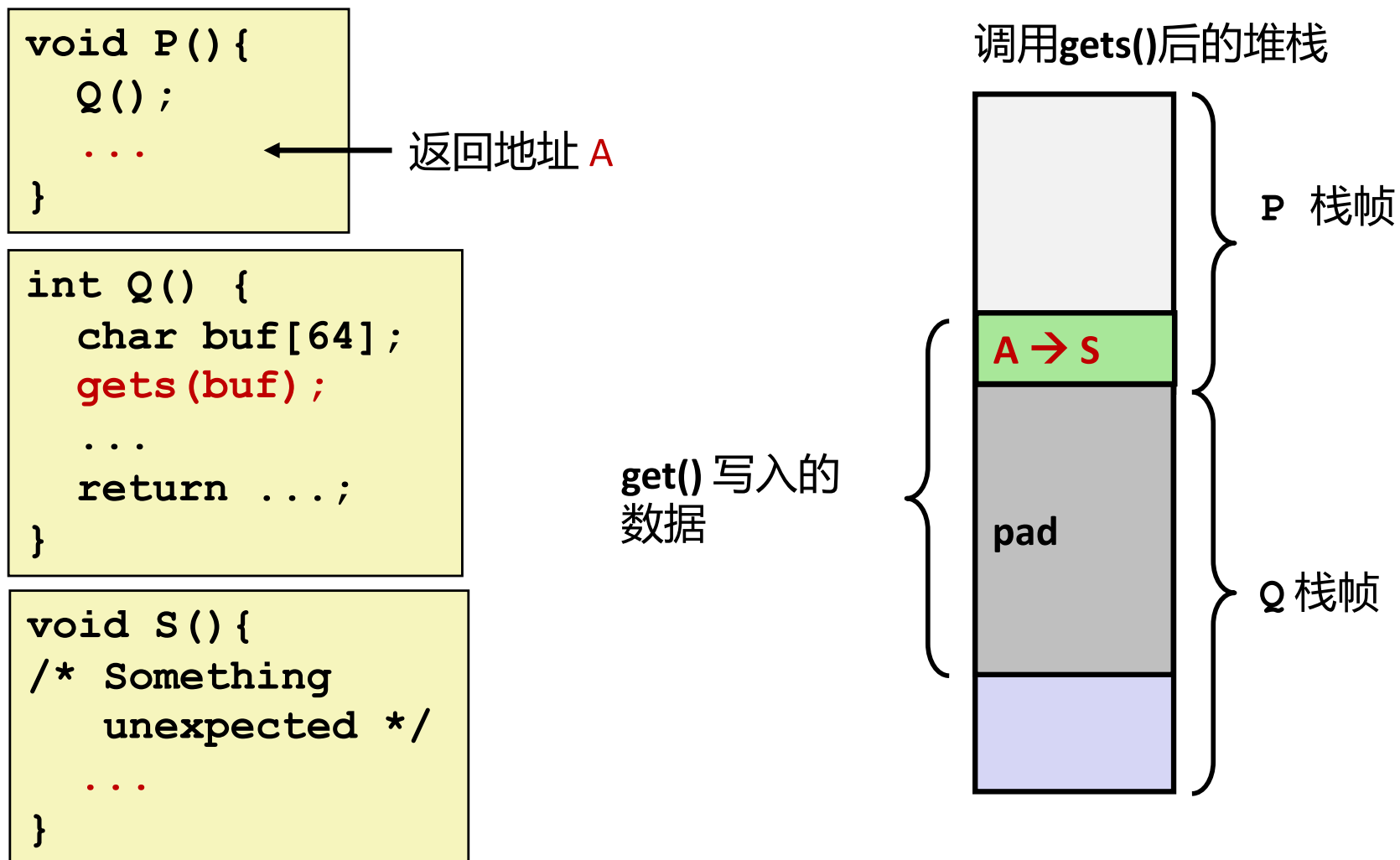
```
. . .  
400600:  mov    %rsp,%rbp  
400603:  mov    %rax,%rdx  
400606:  shr    $0x3f,%rdx  
40060a:  add    %rdx,%rax  
40060d:  sar    %rax  
400610:  jne    400614  
400612:  pop    %rbp  
400613:  retq
```

“返回”到了不相关代码 (地址0x400600)

发生了很多事，但没有破坏程序的关键状态

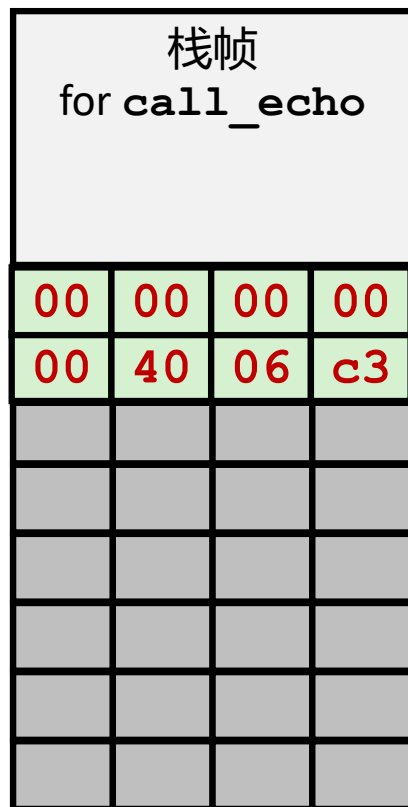
最终执行 retq 返回 main

栈溢出攻击



- 用其他代码段 s 的地址覆盖正常的返回地址 A
- Q 执行 ret 指令时，会跳转到其他代码

构造攻击字符串



```
int echo() {
    char buf[4];
    gets(buf);
    ...
    return ...;
}
```

← %rsp

24 字节

目标代码

```
void smash() {
    printf("I've been smashed!\n");
    exit(0);
}
```

```
00000000004006c8 <smash>:
4006c8:          48 83 ec 08
```

Attack String (Hex)

30	31	32	33	34	35	36	37	38	39	30	31	32	33	34	35	36	37	38	39	30	31	32	33
c8	06	40	00	00	00	00	00																

字符串注入攻击结果

栈帧 for call_echo			
00	00	00	00
00	40	06	c8
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

← %rsp

目标代码

```
void smash() {  
    printf("I've been smashed!\n");  
    exit(0);  
}
```

```
00000000004006c8 <smash>:  
4006c8:          48 83 ec 08
```

Attack String (Hex)

30	31	32	33	34	35	36	37	38	39	30	31	32	33	34	35	36	37	38	39	30	31	32	33
c8	06	40	00	00	00	00	00																

执行栈溢出攻击

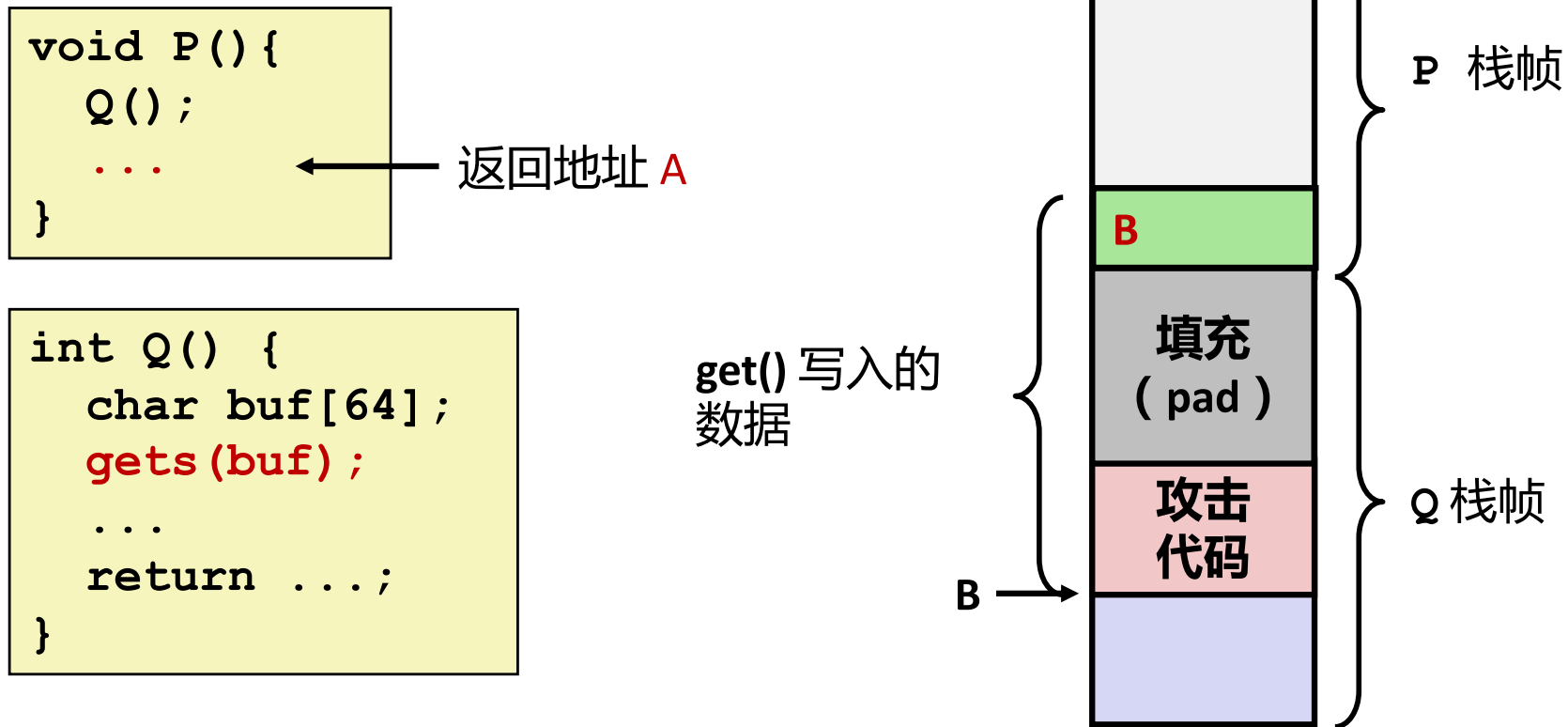
```
linux> cat smash-hex.txt
30 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 37 38 39 30 31 32 33 c8 06 40 00 00 00 00 00
linux> cat smash-hex.txt | ./hexify | ./bufdemo-nsp
Type a string:012345678901234567890123?@
I've been smashed!
```

- 将十六进制序列放入到文件 smash-hex.txt
- 使用 hexify 程序将十六进制数字转换为字符
 - 其中一些是非打印输出的
- 作为输入注入到易受攻击的程序

```
void smash() {
    printf("I've been smashed!\n");
    exit(0);
}
```

```
30 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 37 38 39 30 31 32 33
c8 06 40 00 00 00 00 00
```

代码注入攻击



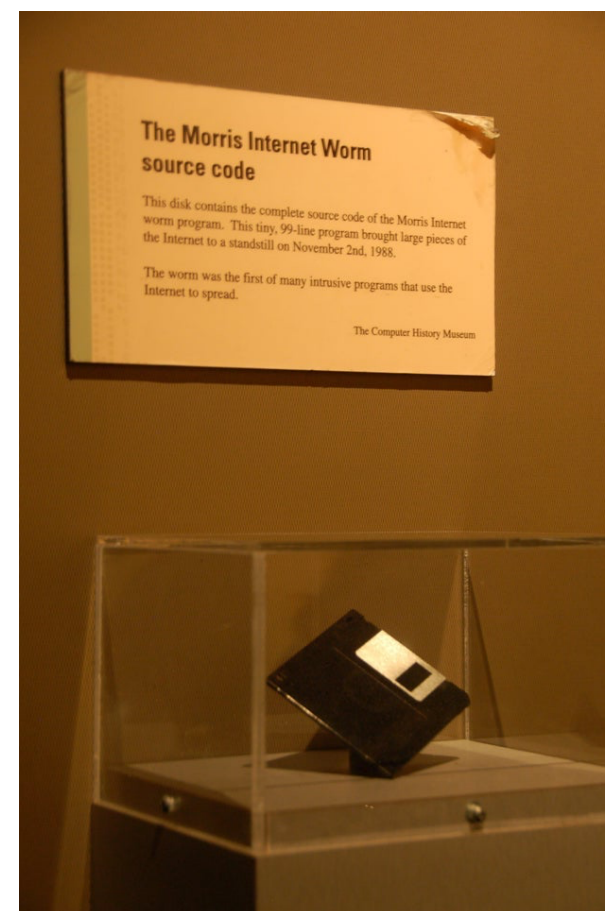
- 输入字符串包含可执行代码的字节表示
- 用缓冲区 **B** 的地址覆盖返回地址 **A**
- **Q** 执行 `ret` 指令时，会跳转到 `exploit` 代码

缓冲区溢出漏洞

- **缓冲区溢出漏洞可以让远程机器在受害机器上执行任意的代码**
- **在实际程序中非常普遍**
 - 程序员总是犯同样的错误 ☹
 - 近些年来一些防护性措施使得这些攻击变得更加困难
- **一些几十年以来的例子**
 - 最初的 “（ 网络蠕虫 ） Internet worm” (1988)
 - “即时通讯战争” (1999)
 - Twilight hack on Wii (2000s)
 - ... 还有很多很多
- **你将在 Attacklab 中学习一些技巧**
 - 希望这些能说服你永远不要在你的程序中留下这样的漏洞！！

危害示例：“蠕虫病毒”(1988)

- 1988年11月2日美国康奈尔大学研究生Robert Morris编写的蠕虫病毒蔓延网络
 - 90行代码
 - 6000台电脑瘫痪（整个互联网的10%）
 - 造成1000万美元以上的损失
 - 3年缓刑，400小时社区劳动，1万美金罚款
 - CMU成立计算机应急响应协调中心CERT/CC



危害示例: “蠕虫病毒”(1988)

- 早期版本的finger服务器使用gets()读取用户参数
 - `finger smartu@whu.edu.cn`
- 蠕虫通过传送虚假参数攻击finger服务器
 - `finger` **“攻击代码 填充 新的返回地址”**
 - “攻击代码”：在目标机器上执行一个`root shell`，获取控制后自我复制
 - “拒绝服务攻击”：耗尽目标系统的资源

蠕虫与病毒

- **蠕虫 (worm) : 一个程序**
 - 可以自己运行
 - 可以将自身的完整工作版本传播到其他计算机
- **病毒 (virus) : 代码**
 - 将自身添加到其他程序
 - 不独立运行
- **两者 (通常) 都旨在在计算机之间传播并造成严重破坏**

如何处理缓冲区溢出攻击

- 避免溢出漏洞
- 采用系统级保护
- 让编译器使用“堆栈金丝雀”

1. 避免代码中的溢出漏洞 (!)

```
/* Echo Line */  
void echo()  
{  
    char buf[4];  
    fgets(buf, 4, stdin);  
    puts(buf);  
}
```

■ 例如，使用限制字符串长度的库例程

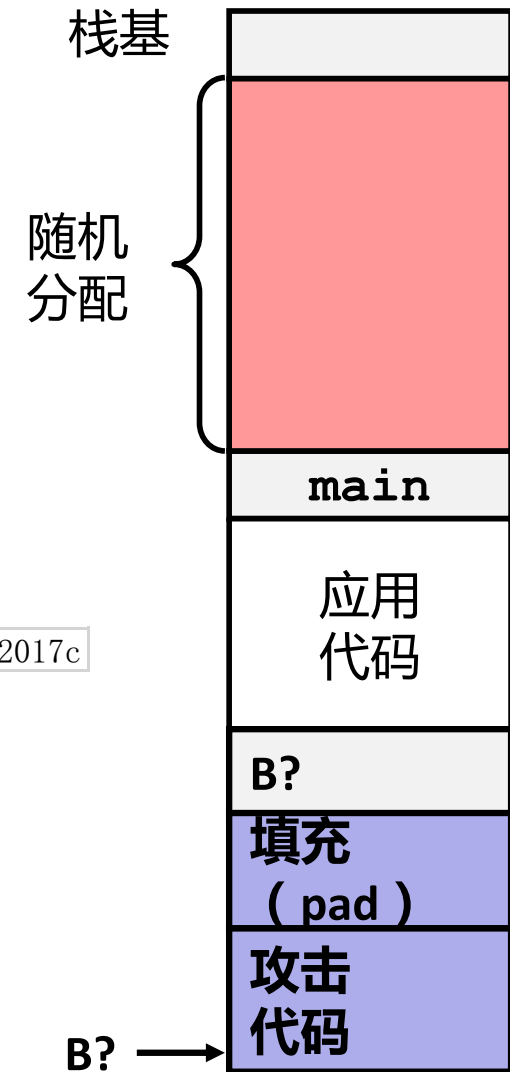
- 使用fgets 而不是 get
- 使用strncpy 而不是 strcpy
- 不要将 scanf 与 %s 转换规范一起使用
 - 使用 fgets 读取字符串
 - 或使用 %ns，其中 n 是合适的整数

2. 系统级保护

■ 栈随机化

- 在程序开始时，在栈上分配一段随机大小的空间
- 整个程序的地址发生位移
- 使黑客难以预测插入代码的地址
- 例如：执行 5 次内存分配代码
 - 每次执行程序，栈的位置都发生改变

local	0x7ffe4d3be87c	0x7fff75a4f9fc	0x7ffeadb7c80c	0x7ffeaea2fdac	0x7ffcd452017c
-------	----------------	----------------	----------------	----------------	----------------



2. 系统级保护

```
bryant - ssh - 87x24

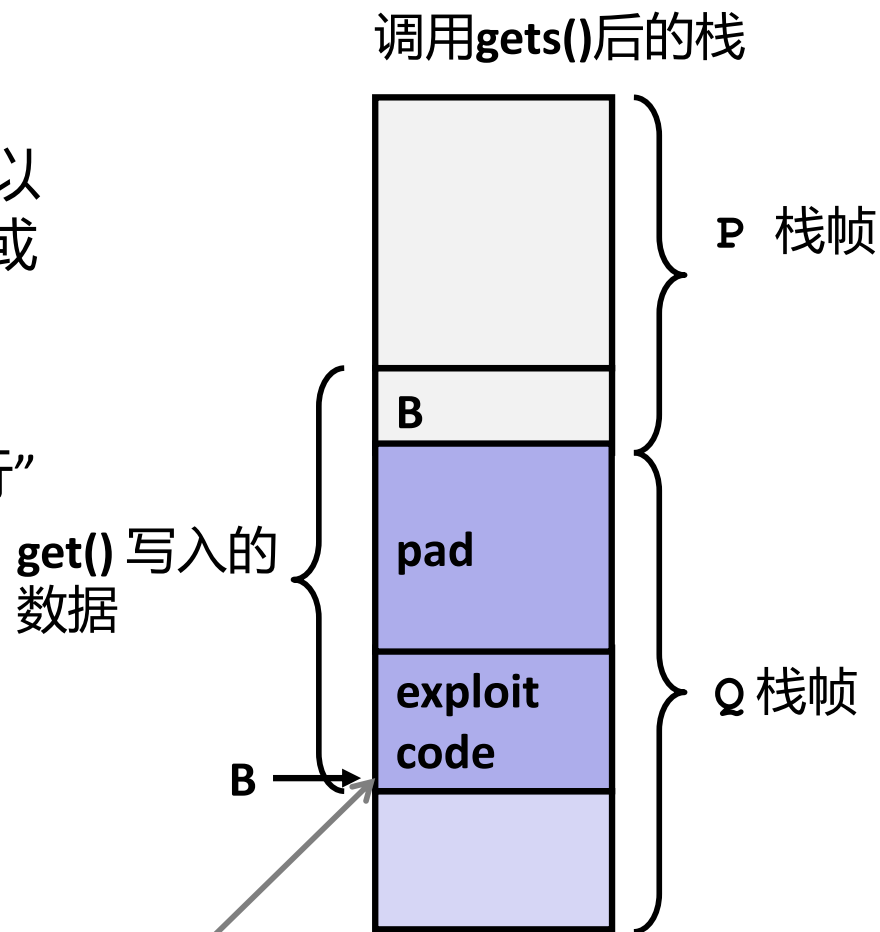
void *p1, *p2, *p3, *p4;
int local = 0;
void *p = malloc(100);

show_pointer((void *) &local, "local");
show_pointer((void *) &global, "global");
show_pointer((void *) p, "heap");
show_pointer((void *) useless, "code");
whaleshark:~/afs/classes/afs213-f15/www/code/09-machine-advanced> ./stack
local  0x7ffe6c6942fc  140730717258492
global 0x60102c       6295596
heap   0x1f5a010      32874512
code   0x400590       4195728
whaleshark:~/afs/classes/afs213-f15/www/code/09-machine-advanced> ./stack
local  0x7ffd66ca9bfc  140726328007676
global 0x60102c       6295596
heap   0xcea010       13541392
code   0x400590       4195728
whaleshark:~/afs/classes/afs213-f15/www/code/09-machine-advanced> ./stack
local  0x7ffe54cc7ddc  140730321108444
global 0x60102c       6295596
heap   0x11f4010      18825232
code   0x400590       4195728
whaleshark:~/afs/classes/afs213-f15/www/code/09-machine-advanced>
```

2. 系统级保护可以提供帮助

■ 不可执行的代码段

- 在传统的 x86 机器中，可以将内存区域标记为“只读”或“可写”
- 可以执行任何可读的文件
- x86-64 添加了明确的“执行”权限
- 栈标记为不可执行



任何执行此代码的尝试都将失败

3. 栈金丝雀



3. 栈金丝雀



3.栈金丝雀

■ 基本思想

- 在缓冲区附近放置一个特殊值（“金丝雀值”）
- 退出函数前检查是否遭到破坏

■ GCC 实现

- `-fstack-protector`
- 现在是默认值（先前已禁用）

```
unix>./bufdemo-sp  
Type a string:0123456  
0123456
```

```
unix>./bufdemo-sp  
Type a string:012345678  
*** stack smashing detected ***
```

受保护缓冲区的汇编代码

echo:

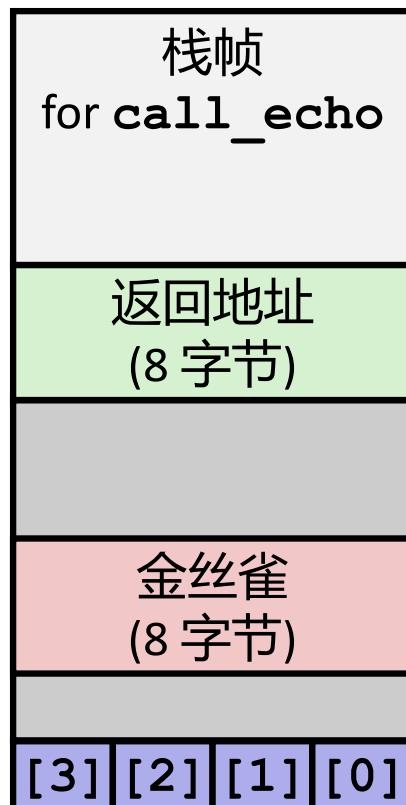
```
40072f:  sub    $0x18,%rsp
400733:  mov    %fs:0x28,%rax
40073c:  mov    %rax,0x8(%rsp)
400741:  xor    %eax,%eax
400743:  mov    %rsp,%rdi
400746:  callq  4006e0 <gets>
40074b:  mov    %rsp,%rdi
40074e:  callq  400570 <puts@plt>
400753:  mov    0x8(%rsp),%rax
400758:  xor    %fs:0x28,%rax
400761:  je     400768 <echo+0x39>
400763:  callq  400580 <__stack_chk_fail@plt>
400768:  add    $0x18,%rsp
40076c:  retq
```

旁注: **%fs:0x28**

- 使用段寻址从内存中读取
- 段是只读的
- 每次程序运行时随机生成的值

设置金丝雀

在调用gets之前



buf ← %rsp

```
/* Echo Line */
void echo()
{
    char buf[4]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

echo:

```
. . .
mov     %fs:0x28, %rax    # Get canary
mov     %rax, 0x8(%rsp)  # Place on stack
xor     %eax, %eax       # Erase register
. . .
```


设置金丝雀

在调用gets之后



buf ← %rsp

```
/* Echo Line */
void echo()
{
    char buf[4]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

输入: 0123456

一些系统：
金丝雀的LSB 0x00
允许输入 01234567

```
echo:
    . . .
    mov     0x8(%rsp),%rax    # Retrieve from stack
    xor     %fs:0x28,%rax    # Compare to canary
    je      .L6              # If same, OK
    call    __stack_chk_fail # FAIL
```

面向返回编程攻击

Return-Oriented Programming Attacks

■ 挑战（针对黑客）

- 栈的随机化使预测缓冲区位置变得困难
- 标记栈不可执行使插入二进制代码变得困难

■ 替代策略

- 使用现成的代码
 - 例如，来自 `stdlib` 库的代码
- 将不同代码段串在一起以实现总体预期结果
- 无法“骗过”栈金丝雀

■ 从gadget s构建程序

- 以 `ret` 指令结尾的指令序列
 - 由单字节 `0xc3` 编码
- 代码位置在每次运行时是固定的
- 代码是可执行的

Gadget示例 #1

```
long ab_plus_c  
  (long a, long b, long c)  
{  
    return a*b + c;  
}
```

```
00000000004004d0 <ab_plus_c>:  
4004d0: 48 0f af fe  imul %rsi,%rdi  
4004d4: 48 8d 04 17  lea (%rdi,%rdx,1),%rax  
4004d8: c3           retq
```

$\text{rax} \leftarrow \text{rdi} + \text{rdx}$

Gadget 地址 = 0x4004d4

- 使用现有功能（函数）的尾端

Gadget示例 #2

```
void setval(unsigned *p) {  
    *p = 3347663060u;  
}
```

<setval>:
4004d9: c7 07 d4 48 89 c7 movl \$0xc78948d4, (%rdi)
4004df: c3 retq

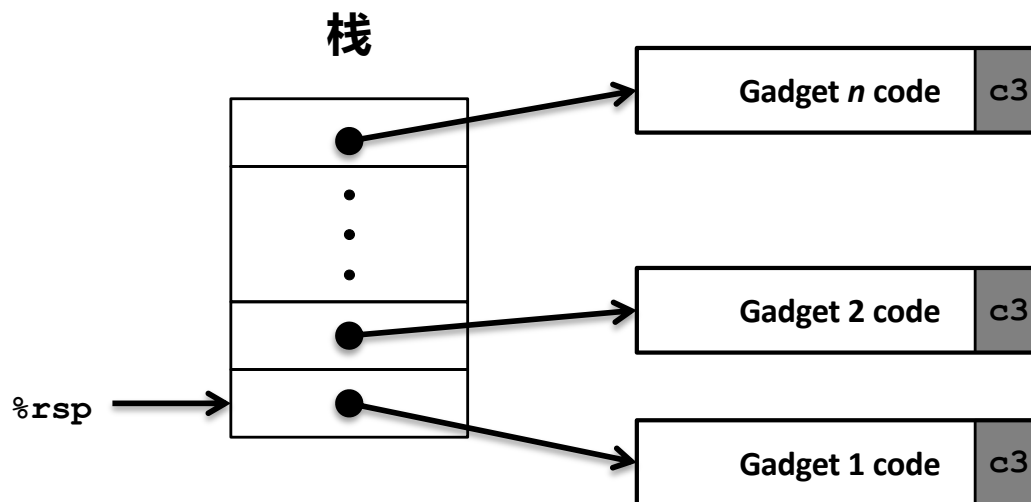
Encodes `movq %rax, %rdi`

`rdi ← rax`

Gadget address = 0x4004dc

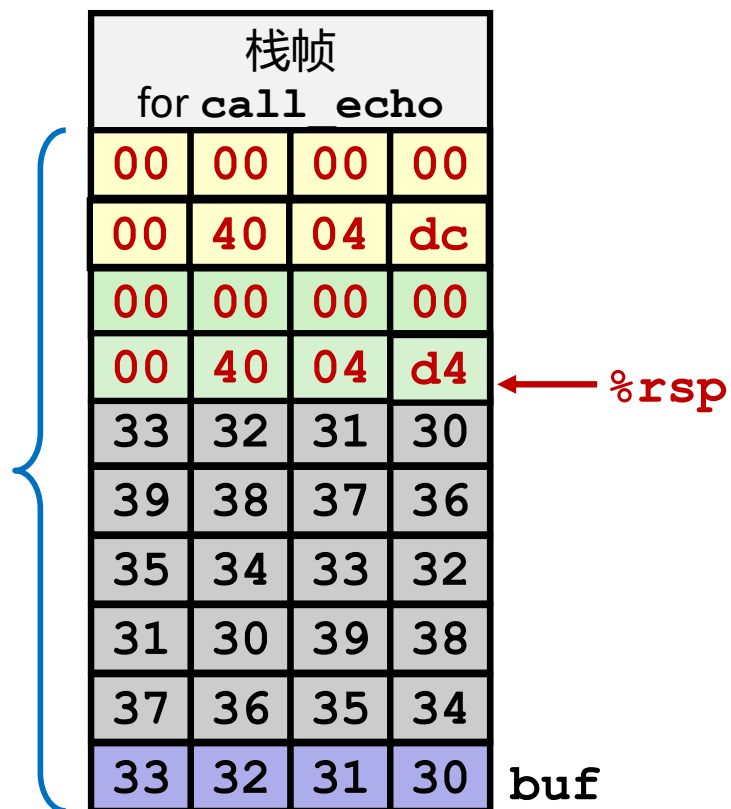
■ 重新利用字节码

ROP 执行



- **用 ret 指令触发**
 - 将开始执行gadget 1
- **每个gadget最终结束后由ret来控制下一个开始**
 - ret : 从栈中弹出地址并跳转到该地址

制作 ROP 攻击字符串



■ gadget #1

■ `0x4004d4` $rax \leftarrow rdi + rdx$

■ gadget #2

■ `0x4004dc` $rdi \leftarrow rax$

■ 组合

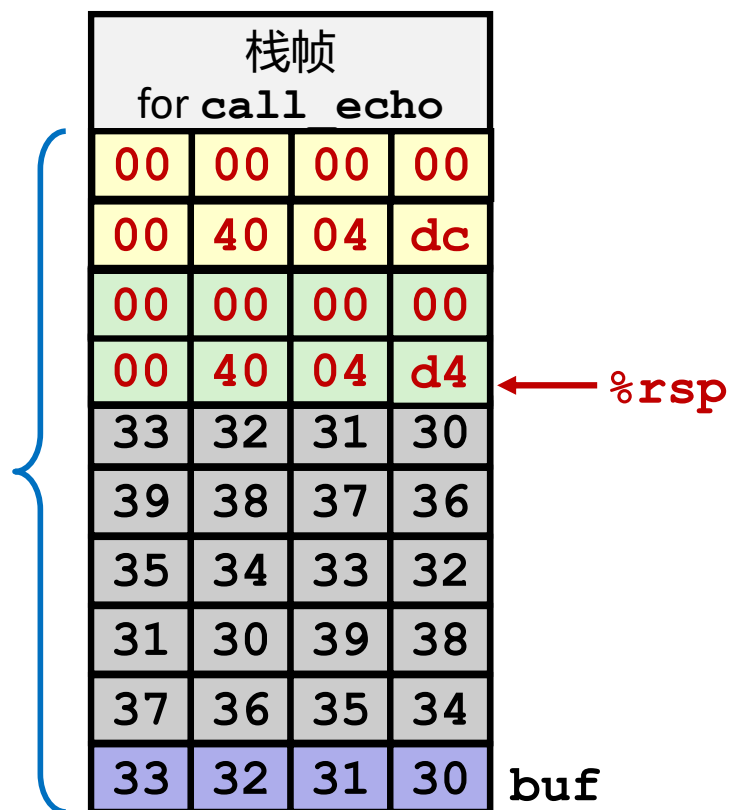
$rdi \leftarrow rdi + rdx$

Attack String (Hex)

30	31	32	33	34	35	36	37	38	39	30	31	32	33	34	35	36	37	38	39	30	31	32	33
d4	04	40	00	00	00	00	00	dc	04	40	00	00	00	00	00	00							

多个gadget会向上破坏栈

当 echo 返回时会发生什么？



1. **echo 执行 ret**
 - 启动Gadget #1
2. **Gadget #1 执行 ret**
 - 启动Gadget #2
3. **Gadget #2 执行 ret**
 - 在某处结束...

多个gadget会向上破坏栈

总结

- 内存布局
- 缓冲区溢出
 - 漏洞
 - 保护
 - 代码注入攻击
 - 返回导向编程